



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

PROYECTO DE CARRERA: INGENIERÍA INFORMÁTICA

UNIDAD CURRICULAR : LENGUAJES Y COMPILADORES

SEMESTRE VII – 2026 – I

# Lenguajes y Gramáticas Formales

Profesor:

Marquez, Felix

Estudiantes:

Frontado, Keiner C.I. 32.059.780

Martínez, Giovanni C.I. 31.782.337

Morales, Edgar C.I. 30.631.492.

Ortega, Mansour C.I. 28.785.297

Ciudad Guayana, junio 2026

## **Introducción**

En el ámbito de la Ingeniería Informática, la Teoría de Lenguajes Formales y Compiladores constituye uno de los pilares fundamentales para comprender cómo las computadoras procesan, estructuran e interpretan instrucciones abstractas. A diferencia de las lenguas naturales, intrínsecamente ambiguas, un lenguaje formal se fundamenta en estructuras matemáticas rigurosas compuestas por alfabetos deterministas, cadenas finitas de símbolos y reglas gramaticales explícitas que eliminan cualquier incertidumbre operativa. El estudio de estos sistemas no solo permite el diseño de lenguajes de programación modernos, sino que define los límites de lo que puede ser procesado computacionalmente a través de la Jerarquía de Chomsky y el modelado de autómatas.

El presente informe tiene como propósito analizar los fundamentos teóricos de las gramáticas formales y demostrar su asombrosa versatilidad mediante aplicaciones multidisciplinarias. En primer lugar, se examinarán los componentes esenciales de un lenguaje (alfabetos, cadenas, clausuras y derivaciones) y los cuatro niveles de complejidad computacional estipulados por Noam Chomsky en 1956. Posteriormente, se abordará un caso práctico de modelado cinemático y vectorial, donde la sintaxis molecular del genoma humano (ADN) y la lógica de los Sistemas de Lindenmayer (Sistemas L) se fusionan para gobernar las trayectorias geométricas de un motor computacional bajo el paradigma de interpretación de Tortuga.

Asimismo, se profundizará en la higiene y optimización de gramáticas para compiladores reales, identificando patologías críticas como la ambigüedad, la recursividad por la izquierda y la necesidad de factorización. Finalmente, el documento valida estos conceptos abstractos mediante el diseño y formalización de un analizador léxico capaz de procesar un subconjunto simplificado de movimientos y capturas en formato PGN (Portable Game Notation) para ajedrez, demostrando de forma empírica la conversión de una expresión regular en un Autómata Finito Determinístico.

## Fundamentos y jerarquía

Entendemos por lenguaje formal como un conjunto de caracteres o símbolos, regidos por reglas gramaticales específicas. Contrasta de los lenguajes naturales debido a que están diseñados para contextos precisos, como lo puede ser la programación, lógica o lingüística, donde la interpretación de las expresiones es estricta y que carece de ambigüedades.

Como todo lenguaje este está compuesto por:

**El Alfabeto ( $\Sigma$ ):** es la base sobre la cual se define un conjunto y no vacío de símbolos básicos. Esto es un requisito indispensable; por ejemplo, el conjunto de números naturales no puede tomarse como alfabeto porque este es infinito, pero si se tomara solo los dígitos del 0 a 9 sí que podría tomarse como alfabeto.

**Cadenas:** se trata de una secuencia finita y ordenada de símbolos extraídos de un alfabeto específico. De esta cadena se consideran aspectos como:

- **Cadena vacía:** Es una secuencia que no contiene ningún símbolo y tiene una importancia fundamental en la teoría de lenguajes. Se denota comúnmente con las letras griegas  $\lambda$  (lambda) o  $\epsilon$  (épsilon).

En un alfabeto binario  $\{0,1\}$ , la cadena vacía representa la ausencia de bits o una "longitud cero" de información.

- **Longitud:** se refiere al número de posiciones ocupadas por los símbolos en la cadena y se denota habitualmente como  $|w|$

- **Ejemplos:**

$|\epsilon|=0$ : Por definición, la cadena vacía siempre tiene longitud cero.

$|\text{hola}|=4$ : Contiene cuatro símbolos del alfabeto español.

$|011011|=6$ : Contiene seis símbolos en un alfabeto binario.

- **Clausura de Kleene ( $\Sigma^*$ ):** Representa el conjunto infinito de todas las posibles palabras que se pueden formar con un alfabeto, incluyendo la cadena vacía. Un lenguaje formal es, matemáticamente, un subconjunto de esta clausura.

- **Ejemplo:** Alfabeto binario  $\Sigma=\{0,1\}$ : Su clausura sería  $\Sigma^*=\{\epsilon,0,1,00,01,10,11,000,001,\dots\}$ .

**Gramática:** conjunto de reglas formales que dictan cómo se pueden combinar los símbolos para generar cadenas válidas. Una gramática formal se compone típicamente de: **símbolos terminales** (elementos básicos del alfabeto), **símbolos no terminales** (variables auxiliares que ayudan en el proceso de generación), **símbolo inicial o axioma** (punto de partida para formar cualquier palabra) y **reglas de producción** (instrucciones que permiten transformar el símbolo inicial en palabras finales reemplazando variables)

**Autómatas:** se tratan de modelos matemáticos capaces de reconocer o generar las cadenas de un lenguaje. Actúan como el mecanismo que decide si una entrada específica pertenece o no al lenguaje basándose en sus estados y transiciones.

**Sintaxis y Semántica:** La sintaxis se refiere a las reglas estructurales (la forma), mientras que la semántica se ocupa del significado o la interpretación de esas estructuras dentro de un sistema computacional o lógico.

## Relación Gramática-Lenguaje

Un lenguaje formal se define a menudo a partir de la gramática que lo produce. Esta relación se puede resumir en los siguientes puntos:

- **Definición:** El lenguaje de una gramática ( $L(G)$ ) es el conjunto de todas las cadenas compuestas únicamente por símbolos terminales que pueden ser derivadas desde el símbolo inicial de esa gramática.
- **Especificación:** La gramática elimina la ambigüedad al establecer reglas rigurosas que no admiten excepciones para la formación de expresiones complejas.
- **Clasificación:** La complejidad de las reglas de una gramática determina el tipo de lenguaje resultante dentro de la Jerarquía de Chomsky (regulares, libres de contexto, sensibles al contexto o irrestrictos).

## El Mecanismo de Generación (Derivación)

El proceso mediante el cual una gramática genera un lenguaje se denomina **derivación**. Este mecanismo puede representarse visualmente mediante árboles de derivación, donde la raíz es

el símbolo inicial y las hojas son los símbolos terminales que forman la cadena final.

Funciona de la siguiente manera:

1. **Punto de Partida (Símbolo Inicial):** El proceso comienza siempre con un símbolo distinguido llamado axioma o símbolo inicial (S), que representa la unidad más grande del lenguaje.
2. **Reglas de Producción:** Son instrucciones de sustitución que permiten transformar una variable en una cadena de otros símbolos (terminales o variables). Tienen la forma general  $A \rightarrow \alpha$ .
3. **Proceso Iterativo de Sustitución:**
  - Se elige una cadena que contenga una variable.
  - Se aplica una regla de producción para reemplazar esa variable por la secuencia correspondiente.
  - Este paso se repite recursivamente hasta que la cadena resultante contenga únicamente símbolos terminales.
4. **Finalización:** Una vez que no quedan variables en la cadena, se ha generado una "palabra" válida del lenguaje.

## Jerarquía de Chomsky

Establecida por Noam Chomsky en 1956, la jerarquía de Chomsky es un sistema de clasificación de los lenguajes formales basado en su complejidad computacional y la capacidad generativa de las gramáticas que los producen. Esta jerarquía organiza los lenguajes en cuatro niveles o tipos, los cuales mantienen una relación de inclusión propia; es decir, cada nivel superior (de menor numeración) contiene a los niveles inferiores.

La clasificación se determina fundamentalmente por el tipo de restricciones aplicadas a las reglas de producción de la gramática. A continuación se detallan los cuatro tipos según sus restricciones:

### Tipo 0: Gramáticas irrestrictas (Lenguajes recursivamente enumerables)

Es el nivel más general de la jerarquía y sus gramáticas no poseen ninguna restricción en sus reglas de producción.

- **Reglas de producción:** Tienen la forma  $\alpha \rightarrow \beta$ , donde tanto el lado izquierdo como el derecho pueden contener cualquier combinación de símbolos terminales y variables, siempre que el lado izquierdo no sea vacío y contenga al menos una variable.
- **Complejidad:** Tienen la misma capacidad que una máquina de Turing, pudiendo representar cualquier objeto formal computable.
- **Ejemplo:**

- **Objetivo:** Deshacer o "limpiar" marcadores temporales de compilación o realizar sustituciones macro complejas. Por ejemplo, si el compilador encuentra una señal de optimización OPT junto a una función fn, puede colapsar o transformar radicalmente la cadena.

- **Representación BNF Conceptual:**

`<programa> ::= "OPT" <funcion> "ejecutar"`

`"OPT" <funcion> ::= "codigo_optimizado"`

- **Explicación técnica:** En la segunda regla, el lado izquierdo ("OPT" <funcion>) se reduce a un solo elemento ("codigo\_optimizado"). Como la longitud disminuye ( $2 > 1$ ), esta regla viola las leyes del Tipo 1, volviéndola automáticamente una gramática Tipo 0 (recursivamente enumerable), equivalente al poder de procesamiento absoluto de una Máquina de Turing.

### Tipo 1: Gramáticas sensibles al contexto

Estas gramáticas imponen restricciones basadas en la longitud de las cadenas y el entorno de los símbolos.

- **Reglas de producción:** Tienen la forma  $\alpha \rightarrow \beta$  con la restricción de que la longitud de la cadena de salida no puede ser menor que la de entrada ( $|\alpha| \leq |\beta|$ ).
- **Restricción clave:** Un símbolo no terminal solo puede ser reemplazado por una cadena específica si se encuentra dentro de un contexto determinado (rodeado de ciertos símbolos).

- **Reconocedor:** Son aceptados por autómatas de pila linealmente acotados.
- **Ejemplo:**
  - **Objetivo:** Modelar de forma abstracta la declaración previa de una variable. Imaginemos que el símbolo T representa un "Tipo de dato" (como int) y V representa una "Variable". Queremos simular la regla semántica de que una variable V solo se transforma en un valor asignado si está precedida por su tipo T.
  - **Representación BNF Adaptada al Contexto:**

```
<declaracion> ::= "int" <variable>
```

```
"int" <variable> ::= "int" "identificador"
```
  - **Explicación técnica:** Observa el lado izquierdo de la segunda regla: "int" <variable>. El no terminal <variable> no puede convertirse en "identificador" libremente; está condicionado a tener el terminal "int" a su izquierda (su contexto). Además, el lado izquierdo tiene 2 elementos y el derecho tiene 2 (\$2 \leq 2\$), cumpliendo con la restricción de longitud.

## Tipo 2: Gramáticas libres de contexto

En este nivel, las producciones se vuelven mucho más restrictivas y son la base de la mayoría de los lenguajes de programación.

- **Reglas de producción:** Tienen la forma  $A \rightarrow \beta$ , donde el lado izquierdo (A) debe ser únicamente un símbolo no terminal.
- **Restricción clave:** El símbolo no terminal puede ser reemplazado por la cadena de símbolos  $\beta$  independientemente del contexto en el que aparezca.
- **Reconocedor:** Son reconocidos por los autómatas con pila (PDA).
- **Ejemplo:**

- **Objetivo:** Permitir expresiones matemáticas con paréntesis balanceados recursivos (por ejemplo:  $(a + a)$ ). Las expresiones regulares (Tipo 3) son matemáticamente incapaces de contar o validar que los paréntesis se cierren correctamente.

- **Representación en BNF:**

`<expresion> ::= <termino> | <expresion> "+" <termino>`

`<termino> ::= "a" | "(" <expresion> ")"`

- **Explicación técnica:** La regla `<termino> ::= "(" <expresion> ")"` demuestra la recursividad. El compilador necesita un Autómata con Pila para "recordar" cuántos paréntesis abrió y asegurarse de que se cierren en el orden correcto, una tarea típica del analizador sintáctico (parser).

### Tipo 3: Gramáticas regulares (Lenguajes regulares)

Es el nivel más restrictivo y describe la clase de lenguajes formales más simple.

- **Reglas de producción:** Las reglas deben seguir una estructura lineal estricta, generalmente de la forma  $A \rightarrow aB$  o  $A \rightarrow a$  (gramáticas lineales por la derecha).
- **Restricción clave:** La posición de un símbolo en una cadena depende únicamente del símbolo que lo precede inmediatamente, lo que genera dependencias lineales simples.
- **Reconocedor:** Son aceptados por los autómatas de estado finito.
- Ejemplo:
  - **Objetivo:** Validar un nombre de variable que empiece obligatoriamente con una letra minúscula (a, b o c) seguida opcionalmente de un único dígito (1 o 2).
  - **Representación en BNF:**

`<variable> ::= "a" <digito_opc> | "b" <digito_opc> | "c" <digito_opc>`



$\langle \text{digito\_opc} \rangle ::= "1" \mid "2" \mid \epsilon$

- **Explicación técnica:** Nota cómo cada regla de producción pasa inmediatamente a un terminal o a un terminal seguido de un único no terminal (lineal derecha). Cumple estrictamente con la restricción de que un Autómata Finito puede reconocerlo sin necesidad de memoria adicional (pila).

## Derivación y Modelado

La biología revela que el genoma de un organismo vivo está cristalizado como un lenguaje de programación físico: el ácido desoxirribonucleico (ADN). Del mismo modo en que una gramática formal construye palabras válidas encadenando caracteres terminales, la naturaleza lee arreglos de bases nitrogenadas y las somete a operaciones de derivación. Cada secuencia de tres letras terminales actúa como una subcadena con semántica estricta. Ciertas cadenas operan estructuralmente como el axioma “S” de una gramática, emitiendo señales de inicialización (por ejemplo, el análogo de la secuencia ATG), mientras que otras cadenas específicas forzan el final de la derivación sintáctica (como TGA, TAG o TAA).

En 1968, el biólogo teórico Aristid Lindenmayer formuló una variación matemática de las gramáticas generativas conocidas como Sistemas L, diseñadas para simular el asombroso patrón de crecimiento celular y la topología recursiva en plantas multicelulares y cianobacterias. A diferencia de las Gramáticas Libres de Contexto ordinarias, en las cuales las reglas de producción se aplican de forma secuencial e individual, los *Sistemas de Lindenmayer* aplican las reglas de sustitución de manera paralela e indiscriminada sobre todos los no terminales de la cadena en un solo paso temporal, mimetizando el metabolismo simultáneo de las células en los tejidos biológicos.

El trabajo colaborativo posterior entre Przemyslaw Prusinkiewicz y Aristid Lindenmayer en la publicación de 1990 "The Algorithmic Beauty of Plants", propició la maduración final de este modelo geométrico computacional logrando establecer un marco de representación visual al asociar las cadenas textuales resultantes de la gramática a una interfaz gráfica de tortuga, donde los símbolos del abecedario funcionaban como comandos mecánicos de dirección y dibujo para una pluma virtual en un entorno de dos y tres dimensiones. Basándose en este pilar del modelado algorítmico, el presente caso práctico

propone adaptar el alfabeto primigenio del genoma para gobernar las trayectorias de un espacio vectorial de diseño geométrico.

## Modelado Vectorial Espacial

Para demostrar de forma empírica la elasticidad y versatilidad aplicativa de la Teoría de Lenguajes Formales, procederemos a definir una Gramática Libre de Contexto formal ( $G_{\text{dibujo}}$ ), cuyo exclusivo objetivo es instruir a un motor computacional cinemático para dibujar trayectorias espaciales complejas. El alfabeto terminal del lenguaje resultante debe apegarse con absoluta inflexibilidad a la nomenclatura del genoma molecular analizado en la sección anterior:  $\Sigma = \{a, c, g, t\}$ .

Mediante el paradigma lógico de interpretación de Tortuga, se infunde una semántica espacial individual a cada una de las bases nitrogenadas. La asociación cinemática diseñada es la siguiente:

- **a (Adenina/Avanzar):** La unidad de procesamiento avanza linealmente en la dirección de su ángulo actual exactamente por una unidad de distancia. A medida que avanza, la tortuga traza de manera indeleble una arista o línea sólida en el plano.
- **t (Timina/Transición de Retroceso Físico):** La unidad invierte inmediatamente su dirección sin alterar su vector angular rotacional subyacente, y retrocede una unidad geométrica de distancia desplazándose en reverso sin imprimir tinta. Permite "regresar" a un nodo o vértice bifurcación para continuar un patrón dendrítico sin dañar el plano con líneas innecesarias cruzadas.
- **c (Citosina/Curvatura Horaria):** La unidad efectúa una rotación en su propio eje vectorial de giro hacia la derecha (en el sentido de las agujas del reloj) equivalente a un ángulo métrico discreto denominado  $\Theta$ .
- **g (Guanina/Giro Antihorario):** La unidad efectúa una rotación en su propio eje vectorial de giro hacia la izquierda (sentido inverso a las agujas del reloj) regido exactamente por la magnitud angular  $\Theta$ .

Al modificar externamente el ángulo, el comportamiento de las rotaciones **c** y **g** alteran la estructura topológica sin la necesidad de reconfigurar o modificar en lo absoluto las reglas de sustitución y derivaciones terminales del sistema gramatical.

Formalización Estructural de la Gramática  $G_{dibujo}$

La arquitectura generativa unificada se define rigurosamente como:

$$G_{dibujo} = (V_N, \Sigma, P, S)$$

- **Vocabulario No Terminal ( $V_N$ ):**

$$\{S, S_{cuad}, S_{arb}, S_{cub}, S_{tri}, S_{cruz}, K, T, B, H, F, D, R, E, A\}$$

- **Alfabeto Terminal ( $\Sigma$ ):** $\{a, c, g, t\}$
- **Axioma Inicial:**  $S$

El conjunto finito de Reglas de Producción ( $P$ ) se estructura de forma descendente, permitiendo que el axioma original ramifique mediante derivación paralela hacia las variables sintácticas responsables de gobernar topologías específicas. Las operaciones se declaran de la siguiente forma axiomática:

| Categoría Geométrica | Producciones Lógicas (P)                                                                                    | Comportamiento Estructural                                                                                                                                    |
|----------------------|-------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Axioma Maestro       | $S \Rightarrow S_{cuad}   S_{tree}   S_{cube}   S_{tri}   S_{cruz}$                                         | Actúa como selector del tipo arquitectónico.                                                                                                                  |
| I. Cuadrado          | $S_{cuad} \Rightarrow KKKK$<br>$K \Rightarrow ac$                                                           | Genera las cuatro aristas cerradas mediante avance y giros rectos.                                                                                            |
| II. Árbol Fractal    | $S_{arb} \Rightarrow T$<br>$T \Rightarrow aBt$<br>$B \Rightarrow gTccTg \mid H$<br>$H \Rightarrow \epsilon$ | Construye iterativamente un tronco, invoca ramificaciones a la izquierda y derecha separadas por giros de ajuste, retornando recursivamente al eje con '   '. |

|                                        |                                                                                                                                                     |                                                                                                                                                                                                 |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                        |                                                                                                                                                     | y finalizando con crecimiento nulo ( $\epsilon$ ).                                                                                                                                              |
| <b>III. Cubo (Perspectiva Oblicua)</b> | $S_{cub} \Rightarrow FDRE$<br>$F \Rightarrow aggaggagga$<br>$D \Rightarrow gga$<br>$R \Rightarrow caggaggagga$<br>$E \Rightarrow tcatctgatccctggga$ | Renderiza la cara frontal, calcula una ruta isométrica en profundidad, dibuja el plano posterior, y conecta los vértices distantes enlazándolos a través de retrocesos silenciosos escalonados. |
| <b>IV. Triángulo</b>                   | $S_{tri} \Rightarrow aHaHaH$<br>$H \Rightarrow cc$                                                                                                  | Ejecuta tres perfiles lineales separados por rotaciones de doble ángulo compensatorio.                                                                                                          |
| <b>V. Cruz Simétrica</b>               | $S_{cruz} \Rightarrow AAAA$<br>$A \Rightarrow atc$                                                                                                  | Traza una recta, se pliega exactamente sobre el centro del plano vectorial sin dibujar, y rota para ejecutar el próximo cuadrante.                                                              |

## Ejemplos de Derivación

A continuación, se detalla la sustitución derivativa formal y el comportamiento físico de los cinco casos geométricos analizados, exponiendo detalladamente las capacidades del Autómata Libre de Contexto que procesa estos patrones lingüísticos. Se utilizan marcadores posicionales como paréntesis y agrupaciones meramente como una herramienta analítica textual para ilustrar el orden de derivación, ya que el compilador procesa flujos de manera serial estrictamente.

### a) Derivación Cuadrado Ortogonal

**Parámetros Físicos del Entorno:** La variable  $\mathbf{P} = 90^\circ$ .

**Desarrollo Sintáctico Iterativo:**

$$1. S \Rightarrow S_{cuad}$$

(Resolución de axioma)

2.  $\Rightarrow KKKK$   
(Invoca cuatro perfiles estructurales  $K$  )
3.  $\Rightarrow acKKK$   
(Primera iteración: Expande el primer  $K$  en  $ac$ )
4.  $\Rightarrow acacKK$   
(Segunda iteración)
5.  $\Rightarrow acacacK$   
(Tercera iteración)
6.  $\Rightarrow acacacac$   
(Culminación Terminal de la Derivación)

La secuencia terminal sintetizada mediante el motor gramatical se define como *acacacac*. Desde una perspectiva de interpretación cinemática, la unidad vectorial efectúa un desplazamiento lineal para plasmar el segmento base ( $a$ ), ejecuta una deflexión ortogonal hacia la derecha ( $c$ ) y proyecta el perfil lateral ascendente ( $a$ ). Posteriormente, la tortuga ajusta su eje rotacional hacia el dominio interno ( $c$ ) para trazar el dintel superior ( $a$ ), culminando con la ejecución de la arista vertical de clausura ( $a$ ) y un giro compensatorio final que restituye el equilibrio de su orientación inicial a  $0^\circ$ .

## b) Derivación Árbol Fractal con Múltiples Ramas y Hojas

**Parámetros Físicos del Entorno:** La variable  $P = 45^\circ$ , lo cual resulta óptimo para emular el ángulo de divergencia foliar de ciertas familias botánicas en simulaciones de L-systems.

### Desarrollo Sintáctico Iterativo (Simulación a 2 niveles de profundidad):

1.  $S \Rightarrow S_{arb}$
2.  $\Rightarrow T$
3.  $\Rightarrow aBt$   
(Trazo base del tronco principal que habilita la ramificación)
4.  $\Rightarrow a(gTccTg)t$   
(Sustitución lógica de la variable generatriz  $B$ )
5.  $\Rightarrow ag(aBt)ccTgt$   
(Resolución de rama primaria izquierda sustituyendo  $T$ )

$$6. \Rightarrow ag(aHt)ccTgt$$

(Alcanzado el límite arbóreo, se invoca a la variable de hoja  $H$ )

$$7. \Rightarrow ag(a\epsilon t)ccTgt \Rightarrow agatccTgt$$

(Dado que  $\epsilon$  es el vacío neutro, la hoja se resuelve estructuralmente sin avanzar, permitiendo la reducción directa)

$$8. \Rightarrow agatcc(aBt)gt$$

(Resolución de rama primaria derecha sustituyendo la segunda variable  $T$ )

$$9. \Rightarrow agatcc(aHt)gt$$

(Límite arbóreo derecho)

$$10. \Rightarrow agatcca\epsilon tgt \Rightarrow agatccatgt$$

(Cadena Terminal Finalizada)

La decodificación analítica de la secuencia terminal *agatccatgt* materializa la recursividad libre de contexto sin utilizar brackets o comandos de pila externos. La unidad vectorial dibuja el tronco central ascendente ( $a$ ), se orienta en oblicuo hacia la izquierda con una rotación de  $45^\circ$  ( $g$ ) y plasma la rama foliar izquierda ( $a$ ). Posteriormente, el gen esencial de retroceso ( $t$ ) arrastra la pluma exactamente hasta el ápice del tronco. Estando orientada aún hacia la izquierda, debe rotar  $90^\circ$  a la derecha para compensar el déficit e inclinarse  $45^\circ$  hacia el noreste ( $cc$ ). Procede a dibujar la rama foliar derecha ( $a$ ), retrocede velozmente a la bifurcación del tronco ( $t$ ), se endereza a la posición recta vertical ( $g$ ) y retrocede silenciosamente la longitud del tronco principal hasta llegar a las raíces ( $t$ ).

### c) Derivación Cubo en Proyección Oblicua (Perspectiva Cavalier)

**Parámetros Físicos del Entorno:** La variable  $P = 45$ .

**Desarrollo Sintáctico Iterativo:**

$$1. S \Rightarrow S_{cub}$$

$$2. \Rightarrow FDRE$$

(Invocación de Caras: Frontal, Diagonal paralela, Cara Trasera, y Aristas de conexión)

$$3. \Rightarrow (aggaggagga)DRE$$

(Renderizado de marco de Cara Frontal; los pares  $gg$  garantizan giros acumulativos de  $90^\circ$ )

4.  $\Rightarrow aggaggagga(ggga)RE$   
(Transición de desplazamiento a fondo. Los giros en cadena *ggg* acumulan 135°, generando la fuga oblicua)
5.  $\Rightarrow aggaggaggaggga(caggaggagga)E$   
(Trazo de la Cara Trasera que descansa paralela a la primera mediante una realineación de rectificación angular de 45° dada por *c*)
6.  $\Rightarrow aggaggaggaggggacaggaggagga(tcacatctgatccctggga)$   
(Reemplazo final de aristas)

Con una monumental cadena terminal derivada de 34 caracteres biológicos, la tortuga logra ensamblar una estructura dodecaédrica en volumen isométrico. Una de las genialidades formales inherentes a las producciones de este diseño recae en la subcadena de aristas de unión. Estando la tortuga estancada en el vértice final y distante de la cara trasera, la serie iterativa de retrocesos invertidos "t" intercalados con giros le asisten a maniobrar en reversa, navegando ciegamente y en silencio a lo largo de las caras de atrás ya entintadas. Una vez que alcanza un vértice desposeído de anclaje de profundidad, la tortuga efectúa un avance conmutado ("a") hacia la cara frontal análoga conectando la arista vacía, para luego inmediatamente retraer su paso y navegar por la cara posterior hacia la siguiente esquina en conflicto. Esta derivación constituye una proeza operativa de la programación abstracta bajo estados libres de contexto.

#### d) Derivación Triángulo Equilátero

**Parámetros Físicos del Entorno:** La variable **P** = 60°.

**Desarrollo Sintáctico Iterativo:**

1.  $S \Rightarrow S_{tri}$
2.  $\Rightarrow aHaHaH$   
(Tres líneas proyectadas vinculadas a dos giros polares)
3.  $\Rightarrow a(cc)aHaH$   
(Reemplazo del primer módulo rotacional, que ensambla 120°)
4.  $\Rightarrow acca(cc)aH$   
(Segunda iteración)
5.  $\Rightarrow accacca(cc) \Rightarrow accaccacc$

(Cadena Completa)

Para garantizar la simetría de un polígono trilátero de ángulos internos de  $60^\circ$ , el cursor debe maniobrar a través de los suplementos externos de deflexión. La gramática inteligentemente expande la variable de giro  $H$  hacia dos terminales de giro horario continuo ( $cc$ ), acumulando una desviación geométrica rotunda de  $120^\circ$ . A medida que procesa la secuencia resultante *accaccacc*, el motor avanza una hipotenusa constante, traza el doble giro radical y cierra con milimétrica fidelidad en el punto generatriz.

#### e) Derivación Cruz Simétrica o Estrella de Cuatro Puntas

**Parámetros Físicos del Entorno:** La variable  $P = 90^\circ$ .

**Desarrollo Sintáctico Iterativo:**

1.  $S \Rightarrow S_{cruz}$

2.  $\Rightarrow AAAA$

(Invocación de los cuatro apéndices magnéticos estelares)

3.  $\Rightarrow actAAA$

(La variable de apéndice  $A$  se fractura en vector, contracción y giro)

4.  $\Rightarrow atcatcAA$

5.  $\Rightarrow atcatcatcA$

6.  $\Rightarrow atcatcatcatc$

(Derivación Completa)

La construcción de un cruce bidireccional puro representa un reto para mecanismos de trazado ininterrumpido. Sin embargo, empleando las propiedades de la gramática libre de contexto acoplada al motor genético en este modelo, la solución es de una elegancia axiomática suprema. El cursor central lee la "palabra" base de doce símbolos: el operador de base avanza de manera rectilínea marcando el apéndice de la cruz ( $a$ ). Acto seguido, y sin efectuar ningún tipo de giro disonante, la timina ( $t$ ) retrae ininterrumpidamente a la tortuga de espaldas hacia el agujero topológico central. Instalado con firmeza sobre el baricentro, aplica un giro rígido de  $90^\circ$  sobre su eje perpendicular ( $c$ ). Al repetir el procedimiento estocástico en forma de bucle finito para completar las doce iteraciones elementales (*atcatcatcatc*), emerge sobre el plano un signo vital más (+) estatuido con total balance armónico desde su propio ombligo geométrico.



Gracias a la introducción estratégica de un operador capaz de mimetizar el retorno funcional de una memoria de tipo pila por medio de la cinemática negativa en el plano mismo, una simple Gramática Libre de Contexto se consolida como una fortaleza algorítmica de potencialidades aún por descubrir. Esta capacidad de gestionar estructuras jerárquicas mediante el retroceso físico permite que el sistema trascienda las limitaciones de los autómatas finitos, aproximándose a la versatilidad de los Autómatas de Pila (PDA) en un entorno puramente visual. La integración de estos principios biológicos y geométricos demuestra que la recursividad no es solo una abstracción matemática, sino una herramienta de diseño fundamental. Al acoplar la sintaxis del ADN con la lógica generativa de los Sistemas L, se establece un puente entre el código fuente de la vida y el modelado algorítmico, permitiendo la creación de arquitecturas complejas a partir de reglas de producción asombrosamente simples.

## Higiene y optimización de gramáticas

Una gramática con patologías es aquella que, a pesar de ser capaz de describir un lenguaje de programación, en sí misma contiene defectos estructurales en sus reglas de producción, los cuales le vuelven inútil, ineficiente o peligrosa para un compilador real. El diseño de lenguajes requiere que cada línea de código corresponda a una única interpretación posible, en este sentido, las gramáticas mal diseñadas, causan errores porque los compiladores y los intérpretes son sumamente lógicos y deterministas. El analizador sintáctico necesita saber exactamente qué camino tomar y qué estructura tiene el código para traducirlo para la máquina, cuando la gramática tiene fallas estructurales, el compilador se “confunde”, lo que ocasiona que entre en bucles infinitos o interpreta el código de una forma totalmente distinta a la que se tenía prevista.

### Gramática ambigua.

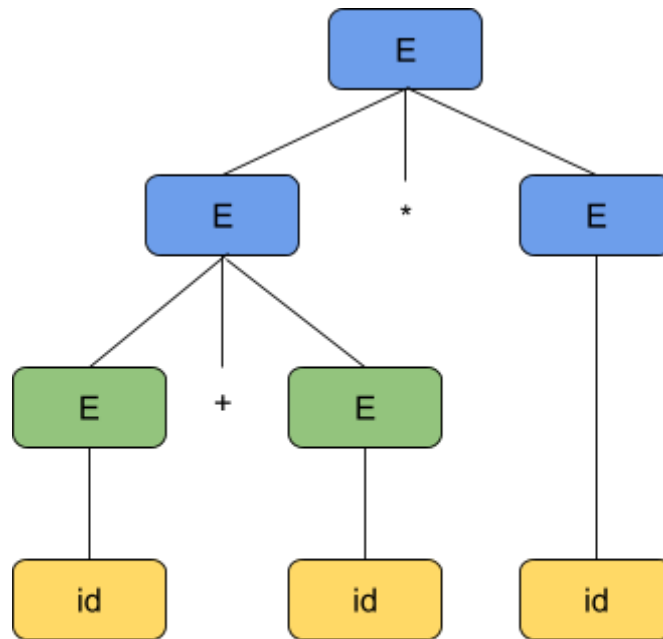
Una gramática es ambigua si una misma cadena puede derivarse de dos o más árboles de derivación distintos, por ejemplo:

Con la gramática:  $E \rightarrow E + E \mid E * E \mid id$ .

La cadena  $id + id * id$  tiene dos árboles de derivación diferentes:

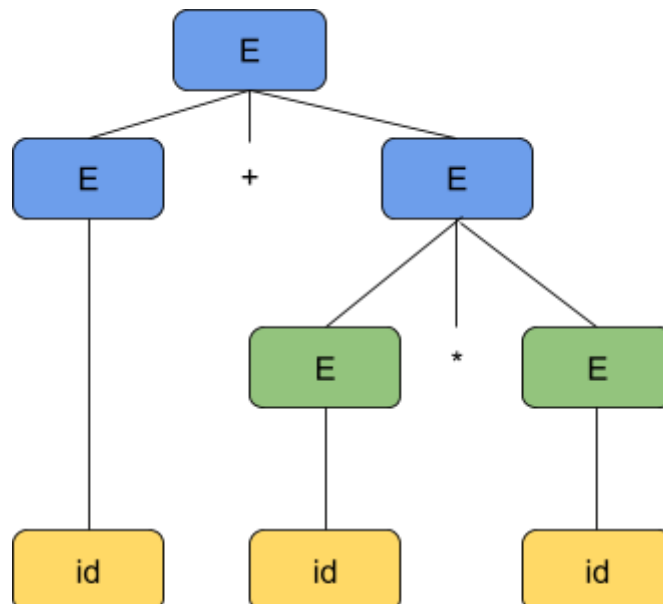
$A \rightarrow A\alpha$ .

Árbol 1:  $(id + id) * id$ ; Suma evaluada primero.



Prioridad de "+" sobre "\*"   
  $(id + id) * id = 2 id * id$

Árbol 2:  $id + (id * id)$ ; Producto evaluado primero.



Prioridad de "\*" sobre "+"   
  $id + (id * id) = id + id^2$

Como se puede observar , la misma cadena:  $id + id * id$  produce resultados semánticamente distintos según el árbol elegido, se considera gramática ambigua porque no define la precedencia de operadores. La solución sería reescribirla con jerarquía explícita:

$$E \rightarrow E + T \mid T.$$

$$T \rightarrow T * F \mid F.$$

$$F \rightarrow id.$$

Donde dejamos le dejamos claro al compilador que, el nivel bajo, es la suma, el nivel medio es el producto y el nivel base es el valor.

### Recursividad por la izquierda.

Una gramática posee recursividad izquierda cuando hay una producción de la forma  $A \rightarrow A\alpha$ , lo que hace que los parsers descendentes entren en bucle infinito, en palabras más sencillas, es cuando el primer símbolo del lado derecho, es el mismo no-terminal del lado izquierdo, por ejemplo:

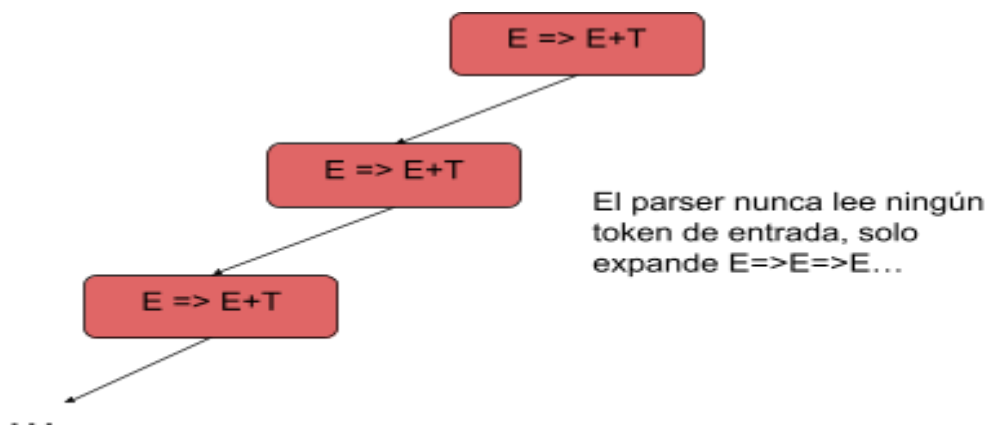
Si la gramática es esta:

$$E \rightarrow E + T \mid T.$$

$$T \rightarrow T * F \mid F.$$

$$F \rightarrow id.$$

Cuando el parser recibe la cadena  $id + id$  e intenta expandir E:



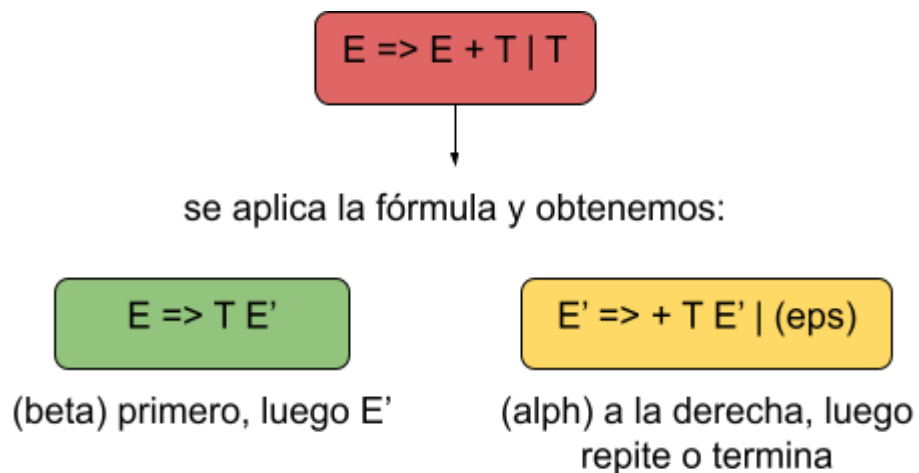
Para entender por qué ocurre este error debemos recordar el funcionamiento del parser LL, el cual mira el siguiente token de la cadena de entrada y decide que producción aplicar. Luego expande de izquierda a derecha; el problema ocurre con  $E \rightarrow E + T$  porque antes de leer cualquier token, el parser ya tiene que expandir E, y para expandir E, tiene que expandir E otra vez, por lo cual nunca llega a leer nada.

Para solucionar este tipo de errores es necesario transformar cualquier regla  $A \rightarrow A\alpha \mid \beta$  agregando dos reglas nuevas:

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \beta.$$

aplicandose a:



(Se usa (alph), (beta) y (eps) para representar a alpha beta y epsilon respectivamente, las cuales no están disponibles en el modo dibujo de google docs)

Donde

$\alpha$  = Parte recursiva original (+ T).

$\beta$  = Base sin recursión (T).

$\epsilon$  = Vacío.

$E'$  = Nuevo no-terminal que absorbe la recursión.

$E'$  puede desaparecer cuando no hay más tokens.

La nueva gramática, pasó de ser:

$$E \rightarrow E + T \mid T.$$

$$T \rightarrow T * F \mid F.$$

$$F \rightarrow id.$$

A ser:

$$E \rightarrow T E'.$$

$$E \rightarrow + T E' \mid \varepsilon.$$

$$T \rightarrow F T'.$$

$$T' \rightarrow * F T' \mid \varepsilon.$$

$$F \rightarrow id.$$

### **Factorización por la izquierda.**

La factorización por la izquierda es una técnica de optimización de gramáticas que se usa principalmente para prepararlas para analizadores sintácticos descendentes, cuando un símbolo no-terminal tiene dos o más reglas de producción que comienzan con el mismo prefijo; al parser le cuesta decidir qué camino tomar sin tener que leer muchos tokens hacia adelante, por ejemplo:

Supongamos las reglas de diseño para una estructura condicional clásica, es decir if-then e if-then-else en un lenguaje de programación de la siguiente manera, donde el símbolo  $S$  representa una sentencia y  $E$  una expresión:

$$S \rightarrow \text{if } E \text{ then } S.$$

$$S \rightarrow \text{if } E \text{ then } S \text{ else } S.$$

$$S \rightarrow a; \text{ donde } a \text{ es cualquier instrucción simple.}$$

Si el analizador sintáctico ve el token *if*, no sabe cuál de las dos primeras reglas aplicar sin ir adelante porque ambas empiezan exactamente igual con el prefijo común:

$$\alpha = \text{if } E \text{ then } S.$$

La regla general de factorización dicta que cuando tienes producciones de la forma:

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2.$$

Se extrae el prefijo común  $\alpha$  y se crea un nuevo no terminal  $A'$  para manejar las diferencias restantes:

$$A \rightarrow \alpha A'.$$

$$A' \rightarrow \beta_1 | \beta_2.$$

Por lo tanto, aplicando la regla general al ejemplo presentado anteriormente de los condicionales:

Prefijo común ( $\alpha$ ): *if E then S*.

Resto de la primera regla ( $\beta_1$ ):  $\varepsilon$  (vacío, no queda nada después).

Resto de la segunda regla ( $\beta_2$ ): *else S*.

El resultado optimizado es el siguiente:

$$S \rightarrow \text{if } E \text{ then } S' | a.$$

$$S' \rightarrow \text{else } S | \varepsilon.$$

De esta manera cuando el compilador encuentra un *if*, sabe que la decisión de si existe un bloque *else* o no se pospone hasta la evaluación de un símbolo  $S'$ , eliminando la necesidad de avanzar en el código para saber si existe el bloque *else*.

### De la Expresión al Autómata (Caso Práctico: Ajedrez)

La notación *PGN* (Portable Game Notation) es un estándar universal diseñado para representar partidas de ajedrez utilizando texto *ASCII* simple, estructurado para ser leído fácilmente tanto por humanos como por programas informáticos. En el contexto de la asignatura de Lenguajes y Compiladores, el reto consiste en construir un modelo matemático (un autómata finito) que actúe como analizador léxico para reconocer y validar si una cadena de texto corresponde a un movimiento legal dentro de este formato. El estándar completo del PGN es muy extenso, ya que no solo registra los movimientos, sino que incluye una sección de metadatos o "etiquetas" (Tag Pairs) con información del evento, números de turno, comentarios encerrados entre llaves {} y variaciones o bifurcaciones de jugadas. Además, el propio texto de los movimientos utiliza la Notación Algebraica Estándar (SAN), la cual tiene reglas complejas para casos especiales como la desambiguación (cuando dos piezas iguales pueden ir a la misma casilla, ej. Rhd4), el enroque (O-O u O-O-O), la promoción de peón (añadiendo el símbolo = seguido de la nueva pieza, ej. =Q) y los sufijos de jaques o mates (+

o #). Para resolver este punto, es estrictamente necesario descartar toda esa complejidad adicional para no generar una explosión de estados inmanejable en el autómata.

La propuesta más sólida para este subconjunto simplificado es limitarse a los movimientos básicos de desplazamiento y captura, definiendo las reglas de la siguiente manera: Movimientos de Peón (no utilizan letra mayúscula inicial y se definen únicamente por la casilla de destino, que consta de una letra a-h y un número 1-8, ej. e4), Movimientos de Piezas Mayores/Menores (requieren una letra inicial mayúscula en inglés K, Q, R, B, o N seguida de la casilla de destino, ej. Nf3), y Capturas Básicas (se intercala la letra minúscula x justo antes de la casilla de destino, ej. xd5 o Nxd4). Al definir este subconjunto, se establece que cualquier cadena aceptada pertenece a las Gramáticas de Tipo 3 en la Jerarquía de Chomsky. Estas gramáticas se procesan de forma eficiente mediante una Expresión Regular (Regex), la cual establece un patrón de búsqueda secuencial que un autómata finito puede validar. Con estas reglas claras, la expresión regular base que gobernará el subconjunto sería equivalente a buscar: opcionalmente una letra de pieza, seguida opcionalmente por una letra x, finalizando obligatoriamente con una letra de la 'a' a la 'h' y un dígito del 1 al 8 ([KQRBN]?x?[a-h]1-8).

### **Notación PGN (Portable Game Notation): Planteamiento para el reconocimiento léxico**

De acuerdo con la especificación oficial *Portable Game Notation Specification and Implementation Guide*, el PGN es un estándar universal diseñado para la representación de datos de partidas de ajedrez utilizando archivos de texto en código ASCII. Este formato está estructurado con el doble propósito de ser fácilmente legible por usuarios humanos y, al mismo tiempo, facilitar su análisis (parsing) y generación por parte de programas informáticos.

Dentro de la estructura de un archivo PGN, las jugadas reales se agrupan en una sección denominada "movetext" (texto de movimientos), la cual emplea estrictamente la Notación Algebraica Estándar (SAN) para registrar cada acción sobre el tablero. Desde la perspectiva de la teoría de Lenguajes y Gramáticas Formales, esta secuencia de caracteres no



es un texto arbitrario, sino un lenguaje formal construido a partir de un alfabeto específico (letras, números y símbolos) donde cada jugada válida representa una "palabra".

El desafío computacional en este punto consiste en procesar y validar dichas palabras. Como explica el análisis *Chess in Regular Expressions*, las jugadas en PGN como e4, xd5 o Nf3 siguen patrones sintácticos bien definidos. Para que una computadora pueda reconocer estas secuencias como válidas, es necesario implementar un analizador léxico sustentado en un autómata finito. Un autómata actúa como un motor matemático de reconocimiento de patrones con memoria limitada, capaz de evaluar si la cadena de caracteres introducida pertenece o no al lenguaje formal del ajedrez.

Sin embargo, el estándar SAN completo presenta una alta complejidad para el procesamiento léxico directo, ya que incluye comentarios anidados, Variaciones de Anotación Recursiva (RAV) e indicadores adicionales (como el sufijo + para jaque o # para mate). Intentar modelar la totalidad de este formato mediante un único autómata finito generaría una explosión de estados inmanejable. Por lo tanto, para aplicar los principios matemáticos de las Gramáticas de Tipo 3 (regulares) y poder diseñar un Autómata Finito Determinístico (AFD) eficiente y libre de ambigüedades, se requiere la abstracción del problema y la definición de un subconjunto restringido del lenguaje.

### **Definición de un subconjunto simplificado del PGN para movimientos básicos**

De acuerdo con el documento oficial *Portable Game Notation Specification and Implementation Guide*, la sección de texto de movimientos (movetext) utiliza la Notación Algebraica Estándar (SAN) para registrar las jugadas. El estándar SAN completo es algorítmicamente complejo, ya que incluye indicadores de jaque (+) o mate (#), reglas de desambiguación de origen de la pieza, enroques (O-O u O-O-O) y promociones de peón (ej. =Q).

Desde la perspectiva de la teoría de lenguajes formales, intentar modelar todas estas variaciones generaría una gramática extensa y un autómata con una explosión de estados inmanejable. Para garantizar la eficiencia y para que el modelo se adhiera a las gramáticas de Tipo 3 (regulares) de la Jerarquía de Chomsky, evitando además ambigüedades o caminos no

determinísticos, se define un subconjunto simplificado restringido exclusivamente a los movimientos básicos de desplazamiento y captura.

Basándose en la especificación oficial SAN, las reglas formales del subconjunto (lenguaje) son las siguientes:

1. Regla de Casillas de Destino: Toda casilla del tablero se identifica obligatoriamente con un carácter para la columna, utilizando una letra minúscula de la a a la h, seguido inmediatamente por un dígito para la fila, del 1 al 8.
2. Movimientos de Peón: Según la sección 8.2.3.2 del estándar, los movimientos de peón no utilizan ninguna letra mayúscula inicial para identificar la pieza. Se definen directamente por su casilla de destino. Ejemplo de movimiento básico: e4.
3. Movimientos de Piezas Mayores/Menores: Las piezas se identifican obligatoriamente con una letra mayúscula inicial en inglés: K (King/Rey), Q (Queen/Reina), R (Rook/Torre), B (Bishop/Alfil) y N (Knight/Caballo). A esta inicial le sigue la casilla de destino de manera contigua. Ejemplo: Nf3.
4. Capturas Básicas: La sección 8.2.3.3 define que, si la jugada resulta en una captura, se debe insertar la letra minúscula x justo antes de la coordenada de destino.

Como explica el analista Brian Dang en su estudio metodológico *Chess in Regular Expressions*, esta reducción de reglas permite establecer un patrón secuencial estricto. Al convertir la letra de la pieza y el símbolo de captura en elementos "opcionales" que preceden a una coordenada obligatoria, el lenguaje resultante es perfectamente compatible con una Expresión Regular básica. Esto cumple con el rigor de ingeniería requerido, ya que garantiza que las cadenas puedan ser validadas por un Autómata Finito Determinístico (AFD) sin caer en bucles o incertidumbres léxicas.

## **Diseño de la Expresión Regular (Regex) y el Autómata Finito Determinístico (AFD) equivalente**

**A) Diseño de la Expresión Regular (Regex)** Como se define en plataformas especializadas de ajedrez, el formato PGN es procesable por computadoras gracias a que

contiene únicamente caracteres ASCII. En su sección de texto de movimientos (movetext), registra la secuencia de las jugadas utilizando la notación algebraica. Para validar léxicamente estas cadenas bajo las reglas de las Gramáticas de Tipo 3 de la Jerarquía de Chomsky, aplicamos un enfoque de reconocimiento de patrones.

Apoyándose en el análisis lógico propuesto por Brian Dang en su artículo *Chess in Regular Expressions*, la conversión de los movimientos básicos de ajedrez a un patrón de búsqueda secuencial nos permite construir una Expresión Regular estricta que evita ambigüedades.

La Expresión Regular que modela de manera exacta el subconjunto simplificado es:  $[KQRBN]?x?[a-h]$ . Su justificación léxica es la siguiente:

- $[KQRBN]?$ : Identifica la inicial de la pieza mayor. El corchete exige la coincidencia con una de esas letras mayúsculas en inglés, y el cuantificador  $?$  establece que su aparición es opcional, dado que los peones no utilizan letra inicial.
- $x?$ : Representa el símbolo de captura. El cuantificador  $?$  indica que también es opcional, ya que no todos los movimientos implican la captura de una pieza rival.
- $[a-h]$ : Representa la columna de destino obligatoria, exigiendo un carácter alfabético minúsculo de la 'a' a la 'h'.
- $[1-8]$ : Representa la fila de destino obligatoria, exigiendo un dígito del '1' al '8' inmediatamente después de la columna.

**B) Diseño del Autómata Finito Determinístico (AFD)** Para procesar esta expresión regular y construir un analizador léxico funcional, se diseña un Autómata Finito Determinístico (AFD), el cual actúa como un motor matemático de reconocimiento de patrones con memoria limitada. Al ser determinístico, se garantiza la "higiene de la gramática" al asegurar que no existan bucles infinitos ni caminos ambiguos durante la compilación.

Formalmente, el autómata se define como la quintupla  $M=(Q,\Sigma,\delta,q_0, F)$ :

- *Alfabeto* ( $\Sigma$ ): Compuesto por los caracteres ASCII válidos extraídos de la notación algebraica estándar: {K, Q, R, B, N, x, a, b, c, d, e, f, g, h, 1, 2, 3, 4, 5, 6, 7, 8}.
- *Conjunto de Estados* ( $Q$ ):  $\{q_0, q_1, q_2, q_3, q_4\}$ .

- *Estado Inicial ( $q_0$ ):*  $q_0$  (Estado de lectura inicial).
- *Estado Final o de Aceptación ( $F$ ):*  $\{q_4\}$  (La cadena representa un movimiento legal válido).

**Función de Transiciones ( $\delta$ ):** El diseño asegura un procesamiento algorítmico eficiente donde, para cada estado, existe a lo sumo un único camino válido por cada símbolo de entrada leído:

1. Desde  $q_0$  (Inicio):
  - Si lee una pieza [KQRBN], transita al estado  $q_1$ .
  - Si lee el símbolo de captura x, transita al estado  $q_2$ .
  - Si lee una columna [a-h], transita al estado  $q_3$ .
2. Desde  $q_1$  (Identificó Pieza):
  - Si lee captura x, transita al estado  $q_2$ .
  - Si lee una columna [a-h], transita al estado  $q_3$ .
3. Desde  $q_2$  (Identificó Captura):
  - Obligatoriamente debe leer una letra de columna [a-h] para transitar al estado  $q_3$ .
4. Desde  $q_3$  (Identificó Columna de destino):
  - Obligatoriamente debe leer un dígito de fila para transitar al estado de aceptación  $q_4$ .

Cualquier otra transición que no esté definida (por ejemplo, leer dos símbolos de captura seguidos o una letra de columna no válida) detendrá la evaluación y enviará la cadena a un estado de rechazo, demostrando que este autómata es una herramienta formal robusta.

## **Conclusión**

La culminación de este informe técnico permite extraer conclusiones fundamentales sobre el impacto práctico y la solidez matemática de la Teoría de Lenguajes y Gramáticas Formales en la informática contemporánea. Se demostró con éxito que la complejidad de un sistema computacional está intrínsecamente ligada a las restricciones de sus reglas de producción. Mientras que las Gramáticas Libres de Contexto (Tipo 2) representan la columna vertebral del análisis sintáctico de la mayoría de los lenguajes de programación por su capacidad recursiva, las Gramáticas Regulares (Tipo 3) se consolidan como la opción óptima por su alta eficiencia y bajo costo computacional cuando se trata de tareas de reconocimiento léxico.

El desarrollo del caso práctico cinemático (G\_dibujo) evidenció que la recursividad y los estados libres de contexto no son meras abstracciones de laboratorio, sino herramientas de diseño geométrico y biológico de inmenso potencial. Al integrar la nomenclatura de las bases nitrogenadas del ADN con la potencia paralela de los Sistemas L y la semántica cinemática negativa (operadores de retroceso), se logró emular el comportamiento de una memoria de tipo pila sobre un plano visual. Esto demuestra que estructuras axiomáticas sumamente simples pueden dar origen a arquitecturas y topologías complejas y balanceadas.

Por otra parte, el análisis de las patologías gramaticales subrayó que la rigurosidad en la etapa de diseño de un lenguaje es obligatoria. Defectos como la ambigüedad o la recursividad izquierda invalidan el determinismo de los analizadores sintácticos descendentes, induciendo al compilador a bucles infinitos o interpretaciones semánticas erróneas. Técnicas de higiene como la factorización y la reescritura con jerarquía explícita eliminan estas ineficiencias de raíz. Finalmente, la simplificación y modelado del estándar PGN de ajedrez ratificó que, ante problemas complejos con riesgo de explosión de estados, la abstracción formalizada permite construir autómatas finitos eficientes, robustos y libres de ambigüedad, uniendo la teoría matemática pura con la ingeniería de software aplicada.

## Referencias bibliográficas

- SEndor, S. (n.d.). Standard: Portable Game Notation Specification and Implementation Guide. Szekeres SEndor.  
[https://www.eurochess.hu/alkonyvtarak/portable\\_game\\_notation\\_specification\\_and\\_implementation/](https://www.eurochess.hu/alkonyvtarak/portable_game_notation_specification_and_implementation/)
- Eureka, A. (2025, January 13). Ajedrez para Aprender Notación PGN. Ajedrez Eureka: Juega Y Disfruta Del Ajedrez Online.  
<https://www.ajedrezeureka.com/ajedrez-para-aprender-pgn/>
- Dang, B. (2025, January 22). Chess in regular expressions. Mr. Dang.  
<https://www.mrdang.com/chess-in-regular-expressions/>
- Chess PGN (Portable Game Notation). (n.d.). [Chess.com](https://www.chess.com/terms/chess-pgn).  
<https://www.chess.com/terms/chess-pgn>
- Codemath. (2023). Operaciones con Palabras | Lenguajes Formales II. Youtube. Recuperado de: [https://www.youtube.com/watch?v=MXDI4Ts\\_EZ0](https://www.youtube.com/watch?v=MXDI4Ts_EZ0)
- Codemath. (2023). Lenguajes Formales desde Cero | Palabra, Alfabeto y Clausura de Kleene. Youtube. Recuperado de: [https://www.youtube.com/watch?v=\\_UdVL-84rXc](https://www.youtube.com/watch?v=_UdVL-84rXc)
- Faria, G. (2025). Lenguajes Formales. concepto. Recuperado de:  
<https://concepto.de/lenguajes-formales/>
- Hulatt, L y Freitas, G. (2023). Formal Language computer science. studysmaster.  
Recuperado:  
<https://www.studysmarter.co.uk/explanations/computer-science/theory-of-computation/formal-language-computer-science/>
- Gallego, Ángel J. (2008). La jerarquía de Chomsky y la facultad del lenguaje: consecuencias para la variación y la evolución. Revista Internacional de Filosofía Vol. 27, No. 2.
- Felipe Pérez, A. (2022). Clasificación de lenguajes naturales en la Jerarquía de Chomsky: lenguajes regulares e independientes de contexto. Recuperada de la Universidad de los Andes.  
<https://repositorio.uniandes.edu.co/entities/publication/a2aa8285-ebd9-45c8-b330-b3708d053761>
- Merrill, W. (2021, 19 febrero). Formal language theory meets modern NLP. Recuperado de <https://arxiv.org/abs/2102.10094>